Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la programación y computación 2, 2do. Semestre 2026

PROYECTO 2 - IPC2

202501037 – Angel Rafael Juárez Higueros

Resumen

La facturación electrónica representa un pilar fundamental en la modernización fiscal de Guatemala mediante el régimen de Documentos Tributarios Electrónicos administrado por la Superintendencia de Administración Tributaria. El presente ensayo expone el diseño e implementación de una arquitectura distribuida basada en servicios web bajo el protocolo de transferencia de hipertexto y el paradigma de programación orientada a objetos en lenguaje C#. La postura técnica adoptada sostiene que el desacoplamiento entre la interfaz de usuario en Razor Pages y la lógica de negocio en una interfaz de programación de aplicaciones permite garantizar la escalabilidad, trazabilidad y validación rigurosa de transacciones tributarias. A nivel técnico y económico, esta solución optimiza la verificación del Registro Tributario Unificado mediante el algoritmo matemático de módulo once, la extracción de patrones temporales con expresiones regulares y el almacenamiento persistente en archivos estructurados. Finalmente, se concluye que la automatización de validaciones contables e integridad referencial reduce significativamente los errores operativos y fortalece la transparencia en la recaudación fiscal nacional.

Palabras clave

API REST, Expresiones Regulares, Facturación Electrónica, Persistencia, POO

Abstract

Electronic invoicing represents a fundamental pillar in the fiscal modernization of Guatemala through the Electronic Tax Document regime administered by the Superintendency of Tax Administration. This essay presents the design and implementation of a distributed architecture based on web services under the hypertext transfer protocol and the object-oriented programming paradigm in the C# language. The adopted technical position maintains that decoupling the user interface in Razor Pages from the business logic in an application programming interface ensures scalability, traceability, and rigorous validation of tax transactions. At a technical and economic level, this solution optimizes the verification of the Unified Tax Registry through the modulus eleven mathematical algorithm, the extraction of temporal patterns using regular expressions, and persistent storage in structured files. Finally, it is concluded that the automation of accounting validations and referential integrity significantly reduces operational errors and strengthens transparency in national tax collection.

Keywords

Electronic Invoicing, OOP, Persistence, Regular Expressions, REST API.

Introducción

La transición hacia ecosistemas digitales de recaudación tributaria exige infraestructuras de software capaces de procesar grandes volúmenes de transacciones financieras en tiempo real sin comprometer la integridad de los datos. En este contexto, el problema central radica en cómo estructurar un servicio web desacoplado que valide, autorice y almacene solicitudes de Documentos Tributarios Electrónicos garantizando exactitud aritmética y consistencia en el padrón de contribuyentes. El propósito de este ensayo es analizar la construcción de una solución integral cliente-servidor en la plataforma .NET Core, fundamentada en la programación orientada a objetos, el uso de tipos de datos abstractos y el intercambio de mensajes en formato JSON. A través del documento se examinan los mecanismos de limpieza léxica por expresiones regulares, la verificación de identificadores tributarios y la generación de reportes estadísticos para la toma de decisiones.

Desarrollo del tema

a. Arquitectura Cliente-Servidor y Desacoplamiento mediante el Protocolo HTTP

El diseño de sistemas empresariales modernos para la gestión pública y tributaria requiere una separación estricta de responsabilidades entre la capa de presentación y la capa de procesamiento de reglas de negocio. En la solución desarrollada para simular el servicio de autorización de la Superintendencia de Administración Tributaria, se adoptó una arquitectura distribuida compuesta por dos programas independientes que interactúan exclusivamente a través del protocolo HTTP. Esta decisión arquitectónica responde a la necesidad de permitir que el servicio central de validación pueda ser consumido de manera estandarizada tanto por el simulador web oficial como por clientes externos de auditoría o pruebas automatizadas como Postman.

El primer componente corresponde al Programa Frontend, construido sobre el marco de trabajo ASP.NET Core Razor Pages. Su responsabilidad se limita a ofrecer una interfaz gráfica intuitiva para que el usuario cargue lotes de archivos en formato JSON, ejecute el reinicio del estado del sistema y visualice reportes tabulares, gráficas comparativas y documentos exportables en formato PDF. Al no contener lógica de validación tributaria interna, el cliente web actúa como un consumidor puro que delega el procesamiento pesado al servidor mediante peticiones asíncronas gestionadas por una fábrica de clientes HTTP.

El segundo componente constituye el núcleo transaccional del sistema y se denomina Servicio Backend, implementado como una Web API en .NET Core. Este servicio expone puntos de acceso específicos bajo los verbos estándar GET y POST para recibir flujos de texto en formato JSON, deserializarlos en objetos de dominio, aplicar las reglas fiscales y retornar respuestas estructuradas. Según exponen Gamma, Helm, Johnson y Vlissides (1995), la división de un sistema en subsistemas débilmente acoplados facilita el mantenimiento evolutivo y evita que los cambios en la interfaz visual afecten la integridad de los componentes de negocio.

Asimismo, la implementación del punto de entrada principal de procesamiento fue diseñada con capacidad de inspección estructural sobre el documento JSON recibido. Cuando un cliente envía

un arreglo de objetos hacia la ruta de procesamiento, el controlador analiza las propiedades del flujo entrante para distinguir automáticamente si la petición corresponde a una actualización del Registro Tributario Unificado o a un lote de solicitudes de autorización de facturas. En caso de recibir archivos malformados, objetos sueltos que no respetan la estructura de arreglo o colecciones vacías, el servicio intercepta la excepción y devuelve un código de estado HTTP cuatrocientos indicando la causa exacta del rechazo sin interrumpir la ejecución del servidor.

b. Paradigma de Programación Orientada a Objetos y Estructuras de Datos

La construcción interna del servicio se fundamenta de manera estricta en el paradigma de Programación Orientada a Objetos en lenguaje C#, cumpliendo con los principios de abstracción, encapsulamiento, modularidad y separación de intereses. Para modelar la realidad del dominio tributario, se abstrajeron las entidades principales en dos categorías de clases claramente diferenciadas: los Objetos de Transferencia de Datos y los Modelos de Dominio Persistentes.

Los Objetos de Transferencia de Datos fueron diseñados para representar de forma idéntica los contratos de entrada y salida especificados para la comunicación JSON. Mediante el uso de atributos de serialización nativos de la plataforma .NET, se mapearon nombres de propiedades con espacios y guiones bajos, tales como los campos de identificación del emisor y del receptor o los contadores de respuesta en mayúsculas. Este encapsulamiento asegura que las variaciones en el formato de texto externo no contaminen la nomenclatura limpia de las propiedades utilizadas dentro de los algoritmos de C#.

Por su parte, el manejo de la información en memoria y su interacción algorítmica se apoyaron en Tipos de Datos Abstractos y colecciones genéricas provistas por el lenguaje C#. De acuerdo con Cormen, Leiserson, Rivest y Stein (2009), la elección adecuada de las estructuras de datos determina directamente la complejidad temporal y el rendimiento de un sistema frente a cargas de volumen. Bajo esta premisa, el Registro Tributario Unificado fue implementado utilizando una estructura de Diccionario basada en tablas de dispersión, donde la clave corresponde al número de identificación tributaria normalizado y el valor almacena la razón social del contribuyente.

La ventaja técnica de utilizar un diccionario para el padrón de contribuyentes radica en que las operaciones de búsqueda de existencia, inserción de nuevos registros y actualización de nombres se ejecutan en tiempo constante promedio. Cuando el sistema procesa lotes masivos de cientos o miles de facturas simultáneas, cada factura requiere verificar que tanto el emisor como el receptor existan previamente en el padrón. Si esta verificación se realizara sobre una lista secuencial, el tiempo de respuesta crecería de forma cuadrática; en cambio, con el diccionario en memoria, la validación de existencia es inmediata.

De igual manera, para garantizar la regla de negocio que exige que el código de referencia interno de cada factura sea único en todo el sistema, se implementó un conjunto de dispersión conocido como HashSet. Al iniciar el procesamiento de un lote de autorizaciones, el servicio carga todas las referencias históricas en dicho conjunto y, a medida que evalúa

cada factura del lote entrante, verifica en tiempo constante si la cadena de referencia ya fue utilizada anteriormente en la base de datos o en un elemento previo del mismo mensaje. Finalmente, el historial completo de facturas evaluadas se mantiene en una lista dinámica de objetos de tipo RegistroFactura, lo que facilita las operaciones de filtrado, agrupamiento y ordenamiento cronológico mediante consultas integradas en el lenguaje.

c. Expresiones Regulares y Algoritmo de Validación de NIT en Base al Módulo Once

Uno de los mayores desafíos en la recepción de datos provenientes de sistemas heterogéneos es la presencia de ruido sintáctico y variaciones de formato en las cadenas de texto. En los mensajes de solicitud de autorización, el atributo correspondiente al tiempo de emisión incluye nombres de departamentos, municipios, signos de puntuación y abreviaturas horarias adicionales a la fecha y hora reales de la operación. El enunciado establece que dicha información adicional no debe considerarse un error, sino que debe descartarse conservando únicamente los datos útiles para el procesamiento.

Para resolver este requerimiento sin recurrir a particiones manuales frágiles, se implementó en el servicio validador un motor de extracción basado en expresiones regulares compiladas. Como señala Friedl (2006), las expresiones regulares constituyen una notación formal de autómatas finitos que permite identificar y capturar subcadenas específicas dentro de textos no estructurados con alta precisión. El patrón diseñado localiza secuencias de uno o dos dígitos para el día, seguidos de una barra diagonal, uno o dos dígitos para el mes, una barra diagonal y cuatro dígitos para el año, acompañados por un espacio y la hora en formato de veinticuatro horas desde las cero horas hasta las veintitrés horas con cincuenta y nueve minutos.

Una vez que la expresión regular extrae los grupos correspondientes a la fecha y la hora, el sistema ejecuta una validación de calendario real mediante análisis exacto invariante. Este segundo filtro resulta indispensable para detectar fechas que cumplen con la forma numérica pero son inexistentes en el calendario gregoriano, como el veintinueve de febrero en un año no bisiesto. Si la cadena de tiempo carece de fecha o de hora, o bien representa una fecha calendario inexistente, el registro se contabiliza como descartado por formato y no contamina las estadísticas del padrón. En caso contrario, la fecha se normaliza al estándar de dos dígitos para día y mes, asegurando uniformidad en todos los reportes posteriores.

El segundo pilar de validación sintáctica y matemática recae sobre el Número de Identificación Tributaria tanto del emisor como del receptor. Primero, una expresión regular verifica que la cadena normalizada en mayúsculas contenga entre uno y veinte caracteres numéricos iniciales seguidos de un último carácter verificador que puede ser un dígito del cero al nueve o la letra K. Posteriormente, el servicio ejecuta el algoritmo oficial de Módulo Once para comprobar la autenticidad del identificador.

El procedimiento algorítmico separa el último carácter de la derecha, que ocupa la posición uno y actúa como dígito verificador esperado, del resto del cuerpo numérico del identificador. A continuación, se recorre el cuerpo numérico de derecha a izquierda multiplicando el valor entero de cada dígito por su posición correlativa, comenzando desde la posición dos hasta alcanzar el extremo izquierdo. Todos los productos parciales se acumulan en una sumatoria

total, a la cual se le calcula el residuo de la división entera entre once. Dicho residuo se resta del número once y, al valor resultante de esa sustracción, se le vuelve a aplicar la operación módulo once. Si el resultado final es igual a diez, el algoritmo establece que el carácter verificador válido corresponde a la letra K; si es menor a diez, corresponde al dígito numérico equivalente. Solo cuando el carácter calculado coincide exactamente con el último carácter ingresado, el identificador es catalogado como válido.

d. Reglas Contables, Generación de Correlativos y Persistencia en Archivos

Superadas las validaciones de formato temporal y autenticidad de los identificadores tributarios, el motor de procesamiento evalúa de forma exhaustiva las reglas contables y de integridad referencial de cada documento electrónico. Una característica crítica del diseño implementado es que las validaciones no son excluyentes entre sí durante el diagnóstico de una factura defectuosa. Esto significa que un mismo documento puede presentar simultáneamente un identificador de emisor inválido, un cálculo erróneo del impuesto al valor agregado y una inconsistencia en el monto total; el sistema registra cada una de las banderas de error activadas para alimentar con exactitud el reporte estadístico por fecha, aunque contabiliza el documento como una única factura rechazada en el conteo general.

En el ámbito aritmético, el sistema verifica en primer lugar que el valor base de la factura sea un número decimal estrictamente positivo. Seguidamente, comprueba que el monto declarado en el campo de Impuesto al Valor Agregado coincida con el resultado de multiplicar el valor base por doce centésimos, aplicando un redondeo comercial a dos posiciones decimales hacia el número más alejado de cero cuando la tercera cifra decimal es cinco. Del mismo modo, se valida que el atributo Total sea exactamente igual a la suma aritmética del valor base más el impuesto declarado. Cualquier discrepancia de incluso un centavo activa las banderas de error de cálculo correspondientes e impide la autorización del documento.

Cuando una solicitud supera satisfactoriamente todas las validaciones de existencia en el padrón, validez de identificadores, unicidad de referencia y exactitud contable, el servicio procede a autorizar el documento y emitir su número único de autorización. El código generado consta de dieciséis dígitos numéricos estructurados bajo el formato de cuatro dígitos para el año, dos dígitos para el mes y dos dígitos para el día en que se solicita la autorización en horario de Guatemala, seguidos de un número correlativo de ocho dígitos rellenado con ceros a la izquierda. Para garantizar que el correlativo inicie con el valor uno cada día y no se repita jamás dentro de esa misma jornada, el servicio mantiene un diccionario indexado por la cadena de fecha de autorización que incrementa atómicamente el contador diario por cada factura aprobada.

Toda la información procesada por la API se almacena de forma persistente en la capa de datos mediante archivos locales en formato JSON dentro de un directorio dedicado en el servidor. Se definieron tres archivos independientes para organizar la persistencia: un archivo para el padrón del Registro Tributario Unificado, un archivo para el historial completo de facturas evaluadas con sus códigos de aprobación y banderas de error, y un tercer archivo para resguardar el estado de los correlativos diarios.

El acceso de lectura y escritura a estos archivos está protegido mediante bloques de exclusión mutua en memoria, evitando condiciones de carrera y asegurando que los datos sobrevivan a reinicios del servidor hasta que el usuario invoque explícitamente la operación de inicialización.

e. Generación de Consultas Estadísticas, Visualización Gráfica y Reportes en PDF

El valor analítico del sistema se materializa en los servicios de consulta y reportería que permiten transformar los datos almacenados en información útil para la fiscalización tributaria. El controlador de consultas expone cuatro rutas especializadas que atienden los requerimientos del programa simulador. La primera consulta permite auditar el comportamiento de un contribuyente específico dentro de un rango de fechas, filtrando ya sea sus facturas emitidas o sus facturas recibidas autorizadas, y presentando tanto el desglose diario de documentos y montos como el gran total facturado en el periodo.

La segunda consulta genera un resumen de movimientos de Impuesto al Valor Agregado para una fecha determinada. El algoritmo identifica todos los números de identificación tributaria que participaron como emisores o receptores en facturas aprobadas durante ese día y calcula para cada uno la sumatoria de impuesto emitido frente al impuesto recibido. Por su parte, la tercera consulta evalúa un rango de fechas completo y ofrece dos perspectivas complementarias: el resumen monetario de valores autorizados por día, pudiendo elegir entre montos totales con impuesto o valores netos sin impuesto, y la matriz estadística diaria que desglosa el total de facturas recibidas, los siete tipos de errores posibles, las facturas sin errores y la cantidad de emisores y receptores distintos.

Para enriquecer la interpretación visual de estos indicadores en el Programa Frontend, se integraron representaciones gráficas dinámicas en las vistas de resumen de impuestos y resumen por rango de fechas. Al recibir la respuesta JSON del servidor, la interfaz web inyecta las series de datos en lienzos gráficos que renderizan diagramas de barras comparativos, facilitando la detección inmediata de picos de facturación o asimetrías entre el débito y el crédito fiscal de los contribuyentes. Finalmente, cada módulo de consulta incorpora una funcionalidad de exportación documental que captura el estado de las tablas estadísticas y los gráficos generados para empaquetarlos en un reporte en formato PDF listo para su archivo o impresión oficial.

Conclusiones

La implementación de una arquitectura cliente-servidor desacoplada mediante una API REST en .NET Core y un cliente web en Razor Pages demostró ser un enfoque altamente efectivo para construir servicios tributarios escalables e interoperables. La separación de responsabilidades permitió validar de forma aislada la lógica de negocio a través de peticiones HTTP estándar y garantizar que el sistema responda de manera consistente tanto frente a lotes válidos de gran volumen como ante entradas con errores sintácticos o semánticos. El uso combinado del paradigma de Programación Orientada a Objetos y estructuras de datos eficientes como diccionarios y conjuntos de dispersión resultó determinante para optimizar el tiempo de respuesta del motor de autorizaciones. La verificación de existencia en el Registro Tributario Unificado y la detección de referencias duplicadas en tiempo constante aseguraron

un rendimiento fluido sin degradación al incrementar el tamaño de los archivos de prueba.

La aplicación de expresiones regulares compiladas junto con el algoritmo matemático de Módulo Once proporcionó una barrera robusta de validación de datos. Este mecanismo permitió descartar información irrelevante en los campos de fecha sin generar falsos rechazos y comprobar con exactitud aritmética la autenticidad de identificadores tributarios de hasta veintiún caracteres, incluyendo el manejo del dígito verificador especial K.

Como recomendación para futuras extensiones del proyecto, se sugiere explorar la migración de la capa de persistencia basada en archivos JSON hacia un motor de base de datos relacional o documental distribuido, así como la incorporación de firmas electrónicas criptográficas en los códigos de autorización emitidos para fortalecer aún más la seguridad informática del ecosistema tributario.

Referencias bibliográficas

Cormen, T. H., Leiserson, C. E., Rivest, R. L., y Stein, C. (2009). *Introduction to Algorithms* (3.ª ed.). MIT Press.

Freeman, A. (2022). *Pro ASP.NET Core 6: Develop Cloud-Ready Web Applications Using MVC, Blazor, and Razor Pages* (9.ª ed.). Apress.

Troelsen, A., y Japikse, P. (2022). *Pro C# 10 with .NET 6: Foundational Principles and Practices in Programming* (11.ª ed.). Apress.